

LENGUAJE DE DEFINICIÓN DE DATOS

PostgreSQL es un sistema de gestión de bases de datos relacional libre, liberado bajo licencia BSD, y como muchos otros proyectos Open Source (Código Abierto), su desarrollo es manejado por una comunidad de desarrolladores y organizaciones comerciales. Dicha comunidad es denominada PGDG (PostgreSQL Global Development Group).

Índice de una tabla:

Permite realizar consultas a los registros de forma rápida. Una tabla se puede indexar por uno o varios campos. El objetivo es acelerar la eficacia de una consulta por parte del motor de base de datos.

Hay distintos tipos de índices:

- *La clave primaria (primary key):* una clave primaria es a su vez un índice. En este caso los valores indexados son únicos y no nulos. Puede haber varios campos en una tabla.
- *Index:* crea un índice común, donde los valores no son necesariamente únicos y no nulos. Puede haber varios campos en una tabla.
- *Unique:* crea un índice para los cuales los valores deben ser únicos y diferentes, aparece un mensaje de error si intentamos agregar un registro con un valor ya existente. Permite valores nulos y pueden definirse varios por tabla.

Restricción de Campos Claves (CONSTRAINT):

Se usa para agregar e identificar campos claves (primaria y foránea) dentro de la estructura física de la base de datos. Esto sirve para luego modificar o eliminar una restricción de campo clave.

Restricción CONSTRAINT CHECK:

La restricción check especifica el rango de los valores con los que puede trabajar un campo (por

ejemplo, un campo que contenga la edad de una persona, pero solo se guarde si es mayor de edad).

Restricción CONSTRAINT DEFAULT:

La restricción DEFAULT indica que un campo tendrá un valor por defecto dentro de un registro.

Lenguaje de Definición de Datos.

Sirve para modificar la estructura de los objetos en una base de datos. Las sentencias básicamente son: CREATE, ALTER, DROP, TRUNCATE (se verá en Lenguaje de Manipulación de Datos).

CREATE: Sirve para crear nuevas estructuras dentro del sistema gestor de la base de datos (bases de datos, tablas, índices, usuarios).

Crear un Usuario. Para crear un usuario dentro de PostgreSQL, se hace de la siguiente forma:

```
CREATE USER nombre_usuario WITH PASSWORD 'contraseña';
```

Por ejemplo: Crear un Usuario llamado angelcrespo y contraseña 'angel12345'

```
postgres=# CREATE USER angelcrespo WITH PASSWORD 'angel12345';  
CREATE ROLE  
postgres=#
```

Crear una Base Datos. Para crear una base de datos en PostgreSQL, se hace de la siguiente forma:

```
CREATE DATABASE nombre_base_de_datos;
```

Por ejemplo: se trabajará con una base de datos llamada *AgendaContactos*, que básicamente manejará la información con respecto a contactos y números de teléfono, donde un contacto puede tener uno o varios números de teléfono y un número de teléfono es asociado a un solo contacto.

Contactos: *idcontacto*, *nombre*, *apellido*.

Teléfono: *idtlf*, *número de contacto*, *tipo de número (móvil, trabajo, fijo)*, *idcontacto*.

```
postgres=# CREATE DATABASE AgendaContactos;
CREATE DATABASE
postgres=#
```

Acceder a una base de datos con un Usuario en específico.

```
psql -U nombre_usuario -h localhost nombre_base_de_datos
```

Ejemplo: Se accederá a la base de datos *agendacontactos* con el Usuario *angelcrespo*.

```
postgres@angelc-M2421:/home/angelc$ psql -U angelcrespo -h localhost agendacontactos
Password for user angelcrespo:
psql (9.1.24)
SSL connection (cipher: DHE-RSA-AES256-GCM-SHA384, bits: 256)
Type "help" for help.

agendacontactos=>
```

Crear Tablas. Para crear tablas para una base de datos en PostgreSQL, se hace de la siguiente forma:

```
CREATE TABLE nombre_tabla (nombre_atributos tipo_de_datos,...);
```

Ejemplos: Crear las tablas contactos y teléfono especificadas anteriormente.

```
agendacontactos=> CREATE TABLE contactos (id serial, nombre varchar, apellido varchar);
NOTICE: CREATE TABLE will create implicit sequence "contactos_id_seq" for serial column "contactos.id"
CREATE TABLE
agendacontactos=>
```

```
agendacontactos=> CREATE TABLE telefono (id serial, numero varchar, tipo varchar, idcontactos integer);
NOTICE: CREATE TABLE will create implicit sequence "telefono_id_seq" for serial column "telefono.id"
CREATE TABLE
agendacontactos=>
```

NOTA: el tipo de dato serial permite crear una secuencia de números enteros de forma automática.

Crear índice a una tabla. Para crear un índice en una base de datos en PostgreSQL, se hace de la siguiente forma:

`CREATE INDEX nombre_índice ON nombre_tabla (lista_nombre_campo_índice);`

Ejemplo: se crea como índice el nombre de contacto.

```
agendacontactos=> CREATE INDEX indicenombre ON contactos (nombre);  
CREATE INDEX  
agendacontactos=> |
```

Crear un índice único en una tabla. Para crear un índice único en una base de datos en PostgreSQL, se hace de la siguiente forma:

`CREATE UNIQUE INDEX nombre_índice ON nombre_tabla (lista_nombre_campo_índice);`

Ejemplo: Se crea el índice número de teléfono.

```
agendacontactos=> CREATE UNIQUE INDEX indicenum ON telefono (numero);  
CREATE INDEX  
agendacontactos=> |
```

ALTER: Sirve para agregar, borrar o modificar elementos dentro de la estructura física de la base de datos (campos, restricciones).

Cambiar Nombre de Usuario. Para cambiar el nombre de un usuario una base de datos en PostgreSQL, se hace de la siguiente forma:

`ALTER USER nombre_actual RENAME TO nuevo_nombre;`

Ejemplo: Cambiar el nombre del usuario *angelcrespo* a el nuevo nombre *angelcrespo71*.

```
postgres=# ALTER USER angelcrespo RENAME TO angelcrespo71;  
NOTICE: MD5 password cleared because of role rename  
ALTER ROLE  
postgres=# |
```

Cambiar Contraseña de Usuario. Para cambiar la contraseña de un usuario una base de datos en PostgreSQL, se hace de la siguiente forma:

`ALTER USER nombre_usuario WITH PASSWORD 'nueva_contraseña';`

Ejemplo: Cambiar contraseña de usuario a 12345678.

```
postgres=# ALTER USER angelcrespo71 with PASSWORD '12345678';  
ALTER ROLE
```

Cambiar el nombre de una Base datos. Para cambiar el nombre de la base de datos en PostgreSQL, se hace de la siguiente forma:

`ALTER DATABASE nombre_actual RENAME TO nuevo_nombre;`

Ejemplo:

```
postgres=# ALTER DATABASE agendacontactos RENAME TO agendacontacto;  
ALTER DATABASE
```

Cambiar el nombre de una Tabla. Para cambiar el nombre de una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

`ALTER TABLE nombre_actual RENAME TO nuevo_nombre;`

Ejemplo:

```
agendacontacto=> ALTER TABLE contactos RENAME TO contacto;  
ALTER TABLE
```

Cambiar el nombre de un campo de una Tabla. Para cambiar el nombre de un campo en una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

`ALTER TABLE nombre_tabla RENAME nombre_actual_campo TO nuevo_nombre;`

Ejemplo:

```
agendacontacto=> ALTER TABLE contacto RENAME id TO idc;  
ALTER TABLE
```

Agregar un campo a una tabla. Para agregar un nuevo campo en una tabla dentro de una base de

datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ADD COLUMN nuevo_campo tipo_de_datos;
```

Ejemplo: Agregar el Campo Alias a la tabla contacto.

```
agendacontacto=> ALTER TABLE contacto ADD COLUMN alias varchar;  
ALTER TABLE
```

Borrar un campo de una tabla. Para borrar el campo de una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla DROP nombre_campo;
```

Ejemplo: Eliminar el Campo Alias de la tabla Contacto.

```
agendacontacto=> ALTER TABLE contacto DROP alias;  
ALTER TABLE
```

Cambiar el tipo de dato de un campo de una tabla. Para cambiar el tipo de dato de un campo de una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ALTER nombre_campo TYPE nuevo_tipo_de_datos;
```

Ejemplo: Cambiar número de teléfono a entero.

```
agendacontacto=> ALTER TABLE telefono ALTER numero TYPE integer;  
ALTER TABLE
```

Agregar una clave primaria a un campo de una tabla. Para agregar una clave primaria para una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ADD CONSTRAINT nombre_de_clave_primaria PRIMARY  
KEY (lista_campo_clave);
```

Ejemplo: Agregar las claves primarias respectivas a las tablas contacto y teléfono.

```
agendacontacto=> ALTER TABLE contacto ADD CONSTRAINT cpid PRIMARY KEY (idc);  
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "cpid" for table "contacto"  
ALTER TABLE
```

```
agendacontacto=> ALTER TABLE telefono ADD CONSTRAINT cpidtlf PRIMARY KEY (id);  
NOTICE: ALTER TABLE / ADD PRIMARY KEY will create implicit index "cpidtlf" for table "telefono"  
ALTER TABLE
```

Agregar una clave foránea a un campo de una tabla. Para agregar una clave foránea en una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ADD CONSTRAINT pk_codigo FOREIGN KEY  
(campo_clave_foránea) REFERENCES nombre_tabla_principal (campo_clave_primaria);
```

Ejemplo: Agregar la clave foránea para la relación entre teléfono y contacto.

```
agendacontacto=> ALTER TABLE telefono ADD CONSTRAINT cfcontacto FOREIGN KEY(idcontactos) REFERENCES contacto(idc) ON  
DELETE CASCADE ON UPDATE CASCADE;  
ALTER TABLE
```

Borrar una CONSTRAINT. Para borrar una restricción de campo clave en una tabla dentro de una base de datos, se hace de la siguiente forma;

```
ALTER TABLE nombre_tabla DROP CONSTRAINT nombre_restricción;
```

Ejemplo: Eliminar todas las restricciones de clave de las tablas.

Clave foránea en teléfono

```
agendacontacto=> ALTER TABLE telefono DROP CONSTRAINT cfcontacto;  
ALTER TABLE
```

Clave primaria contacto

```
agendacontacto=> ALTER TABLE contacto DROP CONSTRAINT cpid;  
ALTER TABLE
```

Clave primaria teléfono

```
agendacontacto=> ALTER TABLE telefono DROP CONSTRAINT cpidtlf;  
ALTER TABLE
```

Agregar un CONSTRAINT CHECK a un campo. Para agregar una restricción check para un campo en una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ADD CONSTRAINT id_check CHECK (condición);
```

Ejemplo: Agregar una restricción al campo de tipos de números de teléfono que solo acepte los valores móvil, fijo o trabajo.

```
agendacontacto=> ALTER TABLE telefono ADD CONSTRAINT tipotlf CHECK (tipo = 'movil' or tipo = 'fijo' or tipo = 'trabajo');  
ALTER TABLE
```

Agregar un CONSTRAINT DEFAULT a un campo. Para agregar un valor por defecto a un campo en una tabla dentro de una base de datos en PostgreSQL, se hace de la siguiente forma:

```
ALTER TABLE nombre_tabla ALTER nombre_campo SET DEFAULT valor_por_defecto;
```

Ejemplo: agregar un valor por defecto, que el campo número de teléfono sea 1111111111.

```
agendacontacto=> ALTER TABLE telefono ALTER numero SET DEFAULT 1111111111;  
ALTER TABLE
```

Cambiar el nombre a un índice. Para cambiar el nombre de un índice en una tabla dentro de una base de datos PostgreSQL, se hace de la siguiente forma:

```
ALTER INDEX nombre_índice RENAME TO nuevo_nombre_índice;
```

Ejemplo: cambiar el identificador del índice de nombre a “buscanombres”.

```
agendacontacto=> ALTER INDEX indicenombre RENAME TO buscanombres;  
ALTER INDEX
```


DROP: Sirve para borrar distintos objetos dentro del sistema gestor de bases de datos (base de datos, tablas, índices, usuarios).

Borrar un índice. Para borrar un índice de un campo dentro de la base de datos PostgreSQL, se realiza de la siguiente forma:

`DROP INDEX nombre_índice;`

Ejemplo: borrar el índice “buscanombres”.

```
agendacontacto=> DROP INDEX buscanombres;  
DROP INDEX
```

Borrar una Tabla. Para borrar una tabla dentro de una base de datos PostgreSQL, se hace de la siguiente forma:

`DROP TABLE nombre_tabla;`

Ejemplo: Borrar tabla de teléfono.

```
agendacontacto=> DROP TABLE telefono;  
DROP TABLE
```

Borrar una Base Datos. Para borrar una base de datos PostgreSQL, se hace de la siguiente forma:

`DROP DATABASE nombre_base_de_datos;`

Ejemplo: Borrar base de datos agendacontacto.

```
postgres=# DROP DATABASE agendacontacto;  
DROP DATABASE
```

Borrar Usuarios. Para borrar un usuario de PostgreSQL, se hace de la siguiente manera:

`DROP USER nombre_usuario;`

Ejemplo: borrar el usuario angelcrespo71

```
postgres=# DROP USER angelcrespo71;  
DROP ROLE
```

TRUNCATE: Sirve para eliminar todos los registros de una tabla de forma rápida.

Borrar Registros de una Tabla.

```
TRUNCATE TABLE nombre_tabla;
```

Comandos a tomar en cuenta: algunos comandos a tomar en cuenta son los siguientes:

sudo su postgres. Ingresar a postgresql una vez tecleada la contraseña.

psql. Por defecto, ingresa como usuario administrador de base de datos postgresql.

\dt. Visualiza las tablas existentes en la base de datos.

\dv. Visualiza las vistas existentes dentro de la base de datos.

\di. Visualiza los índices dentro de la base de datos.

\di nombre_tabla. Visualiza los índices dentro de una tabla en la base de datos.

Select * from nombre_tabla. Visualiza el contenido de una tabla.

Select * from pg_database. Visualiza las bases de datos existentes.